Pack 12 — Diagnostics, Session Quality Telemetry & Support Bundles (real code)

RemoteDesk · Pack 12 — Diagnostics, Session Quality Telemetry & Support Bundles (real code)

> Codex / Claude Code handoff. Copy each file to the path in its heading. Build order: shared → api → web → desktop → tests. Every file is tagged **SAFE_DIRECT_COPY** (new file, drop in) or **REVIEW_REQUIRED** (patch to an existing file — merge by hand).
> Scope: real WebRTC session-quality telemetry (RTT, jitter, packet loss, bitrate, codec), a derived quality score, persisted diagnostic samples linked to Session/Device, and a redacted support-bundle foundation. **Native input execution stays disabled. No live third-party integrations. Support bundles redact tokens / passwords / emails (on request) / Authorization headers / cookies / API keys before anything is saved.**

Zip-ready file tree

```
remotedesk/
  packages/shared/src/
    diagnostics/
      types.ts                          # SAFE_DIRECT_COPY
      qualityScore.ts                   # SAFE_DIRECT_COPY
      supportBundle.ts                  # SAFE_DIRECT_COPY
      redaction.ts                      # SAFE_DIRECT_COPY
      schema.ts                         # SAFE_DIRECT_COPY
      index.ts                          # SAFE_DIRECT_COPY
    index.ts                            # REVIEW_REQUIRED (patch)
  apps/api/
    prisma/schema.prisma                # REVIEW_REQUIRED (patch)
    prisma/migrations/0012_diagnostics/migration.sql   # SAFE_DIRECT_COPY
    src/lib/diagnostics.ts              # SAFE_DIRECT_COPY
    src/repositories/diagnosticsRepository.ts          # SAFE_DIRECT_COPY
    src/services/diagnostics/diagnostics.service.ts    # SAFE_DIRECT_COPY
    src/services/diagnostics/supportBundle.service.ts  # SAFE_DIRECT_COPY
    src/routes/diagnostics.routes.ts    # SAFE_DIRECT_COPY
    src/server.ts                       # REVIEW_REQUIRED (patch)
  apps/web/app/dashboard/
    diagnostics/page.tsx                # SAFE_DIRECT_COPY
    diagnostics/diagnosticsApi.ts       # SAFE_DIRECT_COPY
    diagnostics/QualityBadge.tsx        # SAFE_DIRECT_COPY
    diagnostics/diagnostics.module.css # SAFE_DIRECT_COPY
    sessions/[sessionId]/diagnostics/page.tsx          # SAFE_DIRECT_COPY
    sessions/[sessionId]/diagnostics/SampleTable.tsx   # SAFE_DIRECT_COPY
    page.tsx                            # REVIEW_REQUIRED (patch — add link)
  apps/desktop/src/renderer/src/
    services/diagnosticsReporter.ts     # SAFE_DIRECT_COPY
    services/supportBundleClient.ts     # SAFE_DIRECT_COPY
    services/rtcStatsCollector.ts       # SAFE_DIRECT_COPY
```

```
    session/sessionSetup.ts            # REVIEW_REQUIRED (patch)
  tests/diagnostics/
    qualityScore.test.ts               # SAFE_DIRECT_COPY
    redaction.test.ts                  # SAFE_DIRECT_COPY
    supportBundle.test.ts              # SAFE_DIRECT_COPY
    api-contract.test.ts               # SAFE_DIRECT_COPY
  docs/
    diagnostics-and-support-bundles.md # SAFE_DIRECT_COPY
  generated-remotedesk-pack12-merge-summary.md   # doNotMerge (handoff meta)
  generated-remotedesk-pack12-code-review.md     # doNotMerge (handoff meta)
  generated-remotedesk-pack12-manifest.json      # doNotMerge (handoff meta)
```

Part A — Shared package

`packages/shared/src/diagnostics/types.ts` — SAFE_DIRECT_COPY

```ts
/** Raw WebRTC-derived metrics for a single moment in a session. */
export interface QualityMetrics {
  /** Round-trip time in milliseconds. */
  rttMs: number;
  /** Jitter in milliseconds. */
  jitterMs: number;
  /** Fraction of packets lost, 0..1. */
  packetLoss: number;
  /** Inbound/outbound bitrate in kbps. */
  bitrateKbps: number;
  /** Negotiated video codec, e.g. "VP8", "H264", "AV1". */
  codec: string;
  /** Frames per second, optional (not always available). */
  fps?: number;
}

/** A persisted diagnostic sample, linked to a session (+ device where known). */
export interface DiagnosticSampleDto extends QualityMetrics {
  id: string;
  sessionId: string;
  deviceId: string | null;
  /** Derived 0..100 score for this sample. */
  qualityScore: number;
  /** ISO timestamp the sample was captured client-side. */
  capturedAt: string;
}

/** Input shape posted by the desktop reporter (no id/score; server derives). */
export interface DiagnosticSampleInput extends QualityMetrics {
  sessionId: string;
  deviceId?: string | null;
  capturedAt: string;
}

/** Rolled-up summary for a whole session. */
export interface SessionQualitySummaryDto {
  sessionId: string;
  sampleCount: number;
```

```
  avgQualityScore: number;
  avgRttMs: number;
  avgJitterMs: number;
  avgPacketLoss: number;
  avgBitrateKbps: number;
  worstQualityScore: number;
  firstSampleAt: string | null;
  lastSampleAt: string | null;
}

/** Coarse quality band derived from the score, for UI coloring. */
export type QualityBand = 'excellent' | 'good' | 'fair' | 'poor';

/** Categories of data a support bundle may carry. Content-light by design. */
export interface SupportBundleMetadata {
  /** Client app + os version strings. */
  appVersion: string;
  osVersion: string;
  /** Optional session this bundle is about. */
  sessionId?: string | null;
  deviceId?: string | null;
  /** Free-form, already-redacted key/value diagnostic context. */
  context: Record<string, string>;
  /** Whether the requester asked to also redact email addresses. */
  redactEmails: boolean;
}

export interface SupportBundleDto {
  id: string;
  orgId: string;
  createdById: string;
  metadata: SupportBundleMetadata;
  /** Redaction summary: which categories were scrubbed + counts. */
  redactionReport: RedactionReport;
  createdAt: string;
}

/** What redaction removed, for auditability. No raw secret values are kept. */
export interface RedactionReport {
  redactedKeys: string[];
  totalRedactions: number;
  emailsRedacted: boolean;
}
```

`packages/shared/src/diagnostics/qualityScore.ts` — SAFE_DIRECT_COPY

```
import type { QualityBand, QualityMetrics } from './types';

/**
 * Pure session-quality scoring shared by desktop (preview), API (persisted), and
 * web (display) so the number never differs across surfaces. Returns 0..100.
 *
 * Weighted penalty model. Each factor maps to a 0..1 "badness" then subtracts a
 * weighted share of 100. Tuned for interactive remote-desktop, where RTT and
 * packet loss hurt perceived responsiveness most.
 */
const WEIGHTS = {
```

```
  rtt: 0.3,
  jitter: 0.2,
  packetLoss: 0.35,
  bitrate: 0.15,
} as const;

/** Clamp helper. */
function clamp01(n: number): number {
  if (Number.isNaN(n)) return 1; // unknown = worst, fail toward caution
  return Math.min(1, Math.max(0, n));
}

/** RTT badness: 0ms = perfect, >=300ms = max penalty. */
function rttBadness(rttMs: number): number {
  return clamp01(rttMs / 300);
}

/** Jitter badness: 0ms = perfect, >=100ms = max penalty. */
function jitterBadness(jitterMs: number): number {
  return clamp01(jitterMs / 100);
}

/** Packet-loss badness: 0 = perfect, >=10% loss = max penalty. */
function lossBadness(packetLoss: number): number {
  return clamp01(packetLoss / 0.1);
}

/** Bitrate badness: >=2500kbps = perfect, 0 = max penalty (inverted). */
function bitrateBadness(bitrateKbps: number): number {
  return clamp01(1 - bitrateKbps / 2500);
}

export function computeQualityScore(m: QualityMetrics): number {
  const penalty =
    WEIGHTS.rtt * rttBadness(m.rttMs) +
    WEIGHTS.jitter * jitterBadness(m.jitterMs) +
    WEIGHTS.packetLoss * lossBadness(m.packetLoss) +
    WEIGHTS.bitrate * bitrateBadness(m.bitrateKbps);
  return Math.round(100 * (1 - penalty));
}

/** Map a score to a coarse band for UI coloring. */
export function qualityBand(score: number): QualityBand {
  if (score >= 85) return 'excellent';
  if (score >= 65) return 'good';
  if (score >= 40) return 'fair';
  return 'poor';
}
```

`packages/shared/src/diagnostics/redaction.ts` — SAFE_DIRECT_COPY

```
import type { RedactionReport } from './types';

/**
 * Pure redaction shared by server (authoritative, runs before persistence) and
 * tests. Removes secret-bearing keys and, optionally, email addresses. Never
 * returns the original secret value — replaces with a fixed mask.
```

```
 */
export const REDACTION_MASK = '[REDACTED]';

/** Key names (case-insensitive) whose values are always scrubbed. */
export const SENSITIVE_KEY_PATTERNS: readonly RegExp[] = [
  /authorization/i,
  /cookie/i,
  /set-cookie/i,
  /token/i,
  /password/i,
  /passwd/i,
  /secret/i,
  /api[-_]?key/i,
  /access[-_]?key/i,
  /refresh[-_]?token/i,
  /bearer/i,
  /private[-_]?key/i,
];

const EMAIL_RE = /[a-z0-9._%+-]+@[a-z0-9.-]+¥.[a-z]{2,}/gi;

function isSensitiveKey(key: string): boolean {
  return SENSITIVE_KEY_PATTERNS.some((re) => re.test(key));
}

/**
 * Redact a flat string map. Returns the scrubbed copy + a report. If
 * `redactEmails` is true, email-looking substrings in *values* are masked too.
 */
export function redactContext(
  context: Record<string, string>,
  redactEmails: boolean,
): { redacted: Record<string, string>; report: RedactionReport } {
  const redacted: Record<string, string> = {};
  const redactedKeys: string[] = [];
  let totalRedactions = 0;
  let emailsRedacted = false;

  for (const [key, value] of Object.entries(context)) {
    if (isSensitiveKey(key)) {
      redacted[key] = REDACTION_MASK;
      redactedKeys.push(key);
      totalRedactions += 1;
      continue;
    }
    let v = value;
    if (redactEmails && EMAIL_RE.test(v)) {
      v = v.replace(EMAIL_RE, REDACTION_MASK);
      emailsRedacted = true;
      totalRedactions += 1;
    }
    redacted[key] = v;
  }

  return {
    redacted,
    report: { redactedKeys, totalRedactions, emailsRedacted },
```

```
    };
  }

  /** Convenience guard: throw if any sensitive key survived (defense in depth). */
  export function assertNoSensitiveKeys(context: Record<string, string>): void {
    for (const key of Object.keys(context)) {
      if (isSensitiveKey(key) && context[key] !== REDACTION_MASK) {
        throw new Error(`Unredacted sensitive key in support bundle: "${key}"`);
      }
    }
  }
```

`packages/shared/src/diagnostics/supportBundle.ts` — SAFE_DIRECT_COPY

```
  import type { RedactionReport, SupportBundleMetadata } from './types';
  import { assertNoSensitiveKeys, redactContext } from './redaction';

  /**
   * Build a SAFE support-bundle metadata object from raw input. This is the only
   * sanctioned path: it redacts context, asserts nothing sensitive survives, and
   * returns the scrubbed metadata + report. No secrets are ever retained.
   */
  export function buildSupportBundleMetadata(raw: {
    appVersion: string;
    osVersion: string;
    sessionId?: string | null;
    deviceId?: string | null;
    context?: Record<string, string>;
    redactEmails?: boolean;
  }): { metadata: SupportBundleMetadata; report: RedactionReport } {
    const redactEmails = raw.redactEmails ?? false;
    const { redacted, report } = redactContext(raw.context ?? {}, redactEmails);
    assertNoSensitiveKeys(redacted);

    const metadata: SupportBundleMetadata = {
      appVersion: raw.appVersion,
      osVersion: raw.osVersion,
      sessionId: raw.sessionId ?? null,
      deviceId: raw.deviceId ?? null,
      context: redacted,
      redactEmails,
    };
    return { metadata, report };
  }
```

`packages/shared/src/diagnostics/schema.ts` — SAFE_DIRECT_COPY

```
  import { z } from 'zod';

  export const qualityMetricsSchema = z.object({
    rttMs: z.number().min(0).max(60_000),
    jitterMs: z.number().min(0).max(60_000),
    packetLoss: z.number().min(0).max(1),
    bitrateKbps: z.number().min(0).max(1_000_000),
    codec: z.string().min(1).max(32),
    fps: z.number().min(0).max(240).optional(),
  });
```

```
export const diagnosticSampleInputSchema = qualityMetricsSchema.extend({
  sessionId: z.string().min(1),
  deviceId: z.string().min(1).nullish(),
  capturedAt: z.string().datetime(),
});

export const supportBundleInputSchema = z.object({
  appVersion: z.string().min(1).max(64),
  osVersion: z.string().min(1).max(64),
  sessionId: z.string().min(1).nullish(),
  deviceId: z.string().min(1).nullish(),
  context: z.record(z.string(), z.string()).optional(),
  redactEmails: z.boolean().optional(),
});

export type DiagnosticSampleInputParsed = z.infer<typeof diagnosticSampleInputSchema>;
export type SupportBundleInputParsed = z.infer<typeof supportBundleInputSchema>;
```

`packages/shared/src/diagnostics/index.ts` — SAFE_DIRECT_COPY

```
export * from './types';
export * from './qualityScore';
export * from './redaction';
export * from './supportBundle';
export * from './schema';
```

`packages/shared/src/index.ts` — REVIEW_REQUIRED (patch)

> Add this barrel next to the existing exports.

```
// ...existing exports...
export * from './diagnostics';
```

Part B — API: Prisma schema + migration

`apps/api/prisma/schema.prisma` — REVIEW_REQUIRED (patch)

> Append these three models. They link to the existing `Session` and `Device` models by id. `deviceId` is optional because not every sample has a resolved device. Run `prisma generate` after merging.

```
model DiagnosticSample {
  id           String   @id @default(cuid())
  sessionId    String
  deviceId     String?
  rttMs        Float
  jitterMs     Float
  packetLoss   Float
  bitrateKbps  Float
  codec        String
  fps          Float?
  qualityScore Int
  capturedAt   DateTime
  createdAt    DateTime @default(now())

  session Session @relation(fields: [sessionId], references: [id], onDelete: Cascade)
```

```
  device  Device? @relation(fields: [deviceId], references: [id], onDelete: SetNull)

  @@index([sessionId])
  @@index([deviceId])
  @@index([sessionId, capturedAt])
}

model SessionQualitySummary {
  sessionId        String   @id
  sampleCount      Int      @default(0)
  avgQualityScore  Float    @default(0)
  avgRttMs         Float    @default(0)
  avgJitterMs      Float    @default(0)
  avgPacketLoss    Float    @default(0)
  avgBitrateKbps   Float    @default(0)
  worstQualityScore Int     @default(100)
  firstSampleAt    DateTime?
  lastSampleAt     DateTime?
  updatedAt        DateTime @updatedAt

  session Session @relation(fields: [sessionId], references: [id], onDelete: Cascade)
}

model SupportBundle {
  id              String   @id @default(cuid())
  orgId           String
  createdById     String
  sessionId       String?
  deviceId        String?
  appVersion      String
  osVersion       String
  /// Redacted, content-light metadata. NEVER stores secrets.
  metadata        Json
  redactionReport Json
  createdAt       DateTime @default(now())

  @@index([orgId])
  @@index([sessionId])
  @@index([orgId, createdAt])
}
```

> Also add the back-relations on the existing models (REVIEW_REQUIRED — merge into those model blocks):
> `// model Session { ... add: }`
> `diagnosticSamples DiagnosticSample[]`
> `qualitySummary    SessionQualitySummary?`
>
> `// model Device { ... add: }`
> `diagnosticSamples DiagnosticSample[]`

`apps/api/prisma/migrations/0012_diagnostics/migration.sql` — SAFE_DIRECT_COPY

```
-- DiagnosticSample
CREATE TABLE "DiagnosticSample" (
  "id"           TEXT PRIMARY KEY,
  "sessionId"    TEXT NOT NULL REFERENCES "Session"("id") ON DELETE CASCADE,
  "deviceId"     TEXT REFERENCES "Device"("id") ON DELETE SET NULL,
```

```
  "rttMs"        DOUBLE PRECISION NOT NULL,
  "jitterMs"     DOUBLE PRECISION NOT NULL,
  "packetLoss"   DOUBLE PRECISION NOT NULL,
  "bitrateKbps"  DOUBLE PRECISION NOT NULL,
  "codec"        TEXT NOT NULL,
  "fps"          DOUBLE PRECISION,
  "qualityScore" INTEGER NOT NULL,
  "capturedAt"   TIMESTAMP NOT NULL,
  "createdAt"    TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "DiagnosticSample_sessionId_idx" ON "DiagnosticSample"("sessionId");
CREATE INDEX "DiagnosticSample_deviceId_idx" ON "DiagnosticSample"("deviceId");
CREATE INDEX "DiagnosticSample_sessionId_capturedAt_idx" ON "DiagnosticSample"("sessionId", "capturedAt");

-- SessionQualitySummary
CREATE TABLE "SessionQualitySummary" (
  "sessionId"         TEXT PRIMARY KEY REFERENCES "Session"("id") ON DELETE CASCADE,
  "sampleCount"       INTEGER NOT NULL DEFAULT 0,
  "avgQualityScore"   DOUBLE PRECISION NOT NULL DEFAULT 0,
  "avgRttMs"          DOUBLE PRECISION NOT NULL DEFAULT 0,
  "avgJitterMs"       DOUBLE PRECISION NOT NULL DEFAULT 0,
  "avgPacketLoss"     DOUBLE PRECISION NOT NULL DEFAULT 0,
  "avgBitrateKbps"    DOUBLE PRECISION NOT NULL DEFAULT 0,
  "worstQualityScore" INTEGER NOT NULL DEFAULT 100,
  "firstSampleAt"     TIMESTAMP,
  "lastSampleAt"      TIMESTAMP,
  "updatedAt"         TIMESTAMP NOT NULL DEFAULT now()
);

-- SupportBundle
CREATE TABLE "SupportBundle" (
  "id"              TEXT PRIMARY KEY,
  "orgId"           TEXT NOT NULL,
  "createdById"     TEXT NOT NULL,
  "sessionId"       TEXT,
  "deviceId"        TEXT,
  "appVersion"      TEXT NOT NULL,
  "osVersion"       TEXT NOT NULL,
  "metadata"        JSONB NOT NULL,
  "redactionReport" JSONB NOT NULL,
  "createdAt"       TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "SupportBundle_orgId_idx" ON "SupportBundle"("orgId");
CREATE INDEX "SupportBundle_sessionId_idx" ON "SupportBundle"("sessionId");
CREATE INDEX "SupportBundle_orgId_createdAt_idx" ON "SupportBundle"("orgId", "createdAt");
```

Part C — API: lib, repository, services, routes

`apps/api/src/lib/diagnostics.ts` — SAFE_DIRECT_COPY

```
import {
  computeQualityScore,
  type DiagnosticSampleDto,
  type QualityMetrics,
```

```
  type SessionQualitySummaryDto,
} from '@remotedesk/shared';

/**
 * Pure helpers for diagnostics. No DB access here so summary math is trivially
 * testable and identical between API and any future batch job.
 */

/** Attach a server-derived score to raw metrics. */
export function scoreSample<T extends QualityMetrics>(m: T): T & { qualityScore: number } {
  return { ...m, qualityScore: computeQualityScore(m) };
}

/** Fold a list of persisted samples into a session summary. */
export function summarize(
  sessionId: string,
  samples: DiagnosticSampleDto[],
): SessionQualitySummaryDto {
  if (samples.length === 0) {
    return {
      sessionId,
      sampleCount: 0,
      avgQualityScore: 0,
      avgRttMs: 0,
      avgJitterMs: 0,
      avgPacketLoss: 0,
      avgBitrateKbps: 0,
      worstQualityScore: 100,
      firstSampleAt: null,
      lastSampleAt: null,
    };
  }
  const n = samples.length;
  const sum = samples.reduce(
    (acc, s) => {
      acc.score += s.qualityScore;
      acc.rtt += s.rttMs;
      acc.jitter += s.jitterMs;
      acc.loss += s.packetLoss;
      acc.bitrate += s.bitrateKbps;
      return acc;
    },
    { score: 0, rtt: 0, jitter: 0, loss: 0, bitrate: 0 },
  );
  const times = samples.map((s) => s.capturedAt).sort();
  return {
    sessionId,
    sampleCount: n,
    avgQualityScore: round(sum.score / n),
    avgRttMs: round(sum.rtt / n),
    avgJitterMs: round(sum.jitter / n),
    avgPacketLoss: round(sum.loss / n, 4),
    avgBitrateKbps: round(sum.bitrate / n),
    worstQualityScore: Math.min(...samples.map((s) => s.qualityScore)),
    firstSampleAt: times[0],
    lastSampleAt: times[times.length - 1],
  };
```

```
}

function round(n: number, dp = 1): number {
  const f = 10 ** dp;
  return Math.round(n * f) / f;
}
```

`apps/api/src/repositories/diagnosticsRepository.ts` — SAFE_DIRECT_COPY

```
import type {
  DiagnosticSampleDto,
  SupportBundleDto,
  SupportBundleMetadata,
  RedactionReport,
} from '@remotedesk/shared';
import { prisma } from '../db/prisma';

function sampleToDto(s: any): DiagnosticSampleDto {
  return {
    id: s.id,
    sessionId: s.sessionId,
    deviceId: s.deviceId ?? null,
    rttMs: s.rttMs,
    jitterMs: s.jitterMs,
    packetLoss: s.packetLoss,
    bitrateKbps: s.bitrateKbps,
    codec: s.codec,
    fps: s.fps ?? undefined,
    qualityScore: s.qualityScore,
    capturedAt: s.capturedAt.toISOString(),
  };
}

export const diagnosticsRepository = {
  async insertSample(input: {
    sessionId: string;
    deviceId: string | null;
    rttMs: number;
    jitterMs: number;
    packetLoss: number;
    bitrateKbps: number;
    codec: string;
    fps?: number;
    qualityScore: number;
    capturedAt: Date;
  }): Promise<DiagnosticSampleDto> {
    const s = await prisma.diagnosticSample.create({
      data: { ...input, fps: input.fps ?? null },
    });
    return sampleToDto(s);
  },

  async samplesForSession(sessionId: string): Promise<DiagnosticSampleDto[]> {
    const rows = await prisma.diagnosticSample.findMany({
      where: { sessionId },
      orderBy: { capturedAt: 'asc' },
    });
```

```
    return rows.map(sampleToDto);
  },

  async upsertSummary(summary: {
    sessionId: string;
    sampleCount: number;
    avgQualityScore: number;
    avgRttMs: number;
    avgJitterMs: number;
    avgPacketLoss: number;
    avgBitrateKbps: number;
    worstQualityScore: number;
    firstSampleAt: string | null;
    lastSampleAt: string | null;
  }): Promise<void> {
    const data = {
      sampleCount: summary.sampleCount,
      avgQualityScore: summary.avgQualityScore,
      avgRttMs: summary.avgRttMs,
      avgJitterMs: summary.avgJitterMs,
      avgPacketLoss: summary.avgPacketLoss,
      avgBitrateKbps: summary.avgBitrateKbps,
      worstQualityScore: summary.worstQualityScore,
      firstSampleAt: summary.firstSampleAt ? new Date(summary.firstSampleAt) : null,
      lastSampleAt: summary.lastSampleAt ? new Date(summary.lastSampleAt) : null,
    };
    await prisma.sessionQualitySummary.upsert({
      where: { sessionId: summary.sessionId },
      create: { sessionId: summary.sessionId, ...data },
      update: data,
    });
  },

  async insertSupportBundle(input: {
    orgId: string;
    createdById: string;
    sessionId: string | null;
    deviceId: string | null;
    appVersion: string;
    osVersion: string;
    metadata: SupportBundleMetadata;
    redactionReport: RedactionReport;
  }): Promise<SupportBundleDto> {
    const b = await prisma.supportBundle.create({
      data: {
        orgId: input.orgId,
        createdById: input.createdById,
        sessionId: input.sessionId,
        deviceId: input.deviceId,
        appVersion: input.appVersion,
        osVersion: input.osVersion,
        metadata: input.metadata as unknown as object,
        redactionReport: input.redactionReport as unknown as object,
      },
    });
    return bundleToDto(b);
  },
```

```
  async supportBundleById(id: string, orgId: string): Promise<SupportBundleDto | null> {
    const b = await prisma.supportBundle.findFirst({ where: { id, orgId } });
    return b ? bundleToDto(b) : null;
  },
};

function bundleToDto(b: any): SupportBundleDto {
  return {
    id: b.id,
    orgId: b.orgId,
    createdById: b.createdById,
    metadata: b.metadata as SupportBundleMetadata,
    redactionReport: b.redactionReport as RedactionReport,
    createdAt: b.createdAt.toISOString(),
  };
}
```

`apps/api/src/services/diagnostics/diagnostics.service.ts` — SAFE_DIRECT_COPY

```
import type {
  DiagnosticSampleDto,
  DiagnosticSampleInputParsed,
  SessionQualitySummaryDto,
} from '@remotedesk/shared';
import { computeQualityScore } from '@remotedesk/shared';
import { diagnosticsRepository } from '../../repositories/diagnosticsRepository';
import { summarize } from '../../lib/diagnostics';

export const diagnosticsService = {
  /** Persist a single sample (server derives the score) and refresh the summary. */
  async recordSample(input: DiagnosticSampleInputParsed): Promise<DiagnosticSampleDto> {
    const qualityScore = computeQualityScore(input);
    const sample = await diagnosticsRepository.insertSample({
      sessionId: input.sessionId,
      deviceId: input.deviceId ?? null,
      rttMs: input.rttMs,
      jitterMs: input.jitterMs,
      packetLoss: input.packetLoss,
      bitrateKbps: input.bitrateKbps,
      codec: input.codec,
      fps: input.fps,
      qualityScore,
      capturedAt: new Date(input.capturedAt),
    });
    await this.refreshSummary(input.sessionId);
    return sample;
  },

  async sessionDiagnostics(sessionId: string): Promise<{
    summary: SessionQualitySummaryDto;
    samples: DiagnosticSampleDto[];
  }> {
    const samples = await diagnosticsRepository.samplesForSession(sessionId);
    return { summary: summarize(sessionId, samples), samples };
  },
```

```
  async refreshSummary(sessionId: string): Promise<void> {
    const samples = await diagnosticsRepository.samplesForSession(sessionId);
    await diagnosticsRepository.upsertSummary(summarize(sessionId, samples));
  },
};
```

`apps/api/src/services/diagnostics/supportBundle.service.ts` — SAFE_DIRECT_COPY

```
import {
  AppError,
  ErrorCodes,
  buildSupportBundleMetadata,
  type SupportBundleDto,
  type SupportBundleInputParsed,
} from '@remotedesk/shared';
import { diagnosticsRepository } from '../../repositories/diagnosticsRepository';

export const supportBundleService = {
  /**
   * Create a support bundle. Redaction is mandatory and happens BEFORE anything
   * touches the DB. No secrets are ever persisted.
   */
  async create(
    orgId: string,
    createdById: string,
    input: SupportBundleInputParsed,
  ): Promise<SupportBundleDto> {
    const { metadata, report } = buildSupportBundleMetadata({
      appVersion: input.appVersion,
      osVersion: input.osVersion,
      sessionId: input.sessionId ?? null,
      deviceId: input.deviceId ?? null,
      context: input.context,
      redactEmails: input.redactEmails,
    });
    return diagnosticsRepository.insertSupportBundle({
      orgId,
      createdById,
      sessionId: metadata.sessionId ?? null,
      deviceId: metadata.deviceId ?? null,
      appVersion: metadata.appVersion,
      osVersion: metadata.osVersion,
      metadata,
      redactionReport: report,
    });
  },

  async get(orgId: string, bundleId: string): Promise<SupportBundleDto> {
    const bundle = await diagnosticsRepository.supportBundleById(bundleId, orgId);
    if (!bundle) throw new AppError(ErrorCodes.SUPPORT_BUNDLE_NOT_FOUND);
    return bundle;
  },
};
```

> Add the error code (REVIEW_REQUIRED — patch `packages/shared/src/errors/errorCodes.ts`):
> `SUPPORT_BUNDLE_NOT_FOUND: 'SUPPORT_BUNDLE_NOT_FOUND',`

`apps/api/src/routes/diagnostics.routes.ts` — SAFE_DIRECT_COPY

```
import { Router } from 'express';
import {
  diagnosticSampleInputSchema,
  supportBundleInputSchema,
} from '@remotedesk/shared';
import { requireAuth, type AuthedRequest } from '../middleware/requireAuth.middleware';
import { orgContext, type OrgRequest } from '../middleware/orgContext.middleware';
import { diagnosticsService } from '../services/diagnostics/diagnostics.service';
import { supportBundleService } from '../services/diagnostics/supportBundle.service';

const h =
  (fn: (req: any, res: any) => Promise<void>) =>
  (req: any, res: any, next: any) =>
    fn(req, res).catch(next);

/**
 * /api/diagnostics
 *   POST /samples                      -> record a quality sample
 *   GET  /sessions/:sessionId          -> summary + samples for a session
 *   POST /support-bundles              -> create a redacted support bundle
 *   GET  /support-bundles/:bundleId     -> fetch bundle metadata (redacted)
 */
export const diagnosticsRoutes = Router()
  .post('/samples', requireAuth, h(async (req, res) => {
    const input = diagnosticSampleInputSchema.parse(req.body);
    res.status(201).json(await diagnosticsService.recordSample(input));
  }))
  .get('/sessions/:sessionId', requireAuth, h(async (req, res) => {
    res.json(await diagnosticsService.sessionDiagnostics(req.params.sessionId));
  }))
  .post('/support-bundles', requireAuth, orgContext, h(async (req, res) => {
    const input = supportBundleInputSchema.parse(req.body);
    const r = req as OrgRequest & AuthedRequest;
    res.status(201).json(await supportBundleService.create(r.orgId, r.userId, input));
  }))
  .get('/support-bundles/:bundleId', requireAuth, orgContext, h(async (req, res) => {
    const r = req as OrgRequest;
    res.json(await supportBundleService.get(r.orgId, req.params.bundleId));
  }));
```

`apps/api/src/server.ts` — REVIEW_REQUIRED (patch)

> Mount the router next to the other `/api/*` mounts.

```
// ...existing imports...
import { diagnosticsRoutes } from './routes/diagnostics.routes';

// ...inside createServer(), next to the other app.use('/api/...') mounts:
app.use('/api/diagnostics', diagnosticsRoutes);
```

Part D — Desktop: telemetry reporter + support bundle client

`apps/desktop/src/renderer/src/services/rtcStatsCollector.ts` — SAFE_DIRECT_COPY

```
import type { QualityMetrics } from '@remotedesk/shared';

/**
 * Reads getStats() from an RTCPeerConnection and reduces it to QualityMetrics.
 * Pure parsing logic is split out (parseStats) so it can be unit-tested without
 * a live peer connection. Returns null if no usable inbound stream is found.
 */
export async function collectRtcMetrics(
  pc: RTCPeerConnection,
): Promise<QualityMetrics | null> {
  const report = await pc.getStats();
  return parseStats(report);
}

/** Pure: turn an RTCStatsReport-like iterable into QualityMetrics. */
export function parseStats(
  report: Iterable<any> | Map<string, any>,
): QualityMetrics | null {
  let inbound: any | null = null;
  let candidatePair: any | null = null;
  let codecId: string | null = null;

  const entries: any[] = [];
  // Both Map and RTCStatsReport are iterable of [key, value] or value.
  for (const item of report as any) {
    const stat = Array.isArray(item) ? item[1] : item;
    entries.push(stat);
  }

  for (const stat of entries) {
    if (stat.type === 'inbound-rtp' && stat.kind === 'video') {
      inbound = stat;
      codecId = stat.codecId ?? null;
    } else if (stat.type === 'candidate-pair' && stat.state === 'succeeded') {
      candidatePair = stat;
    }
  }
  if (!inbound) return null;

  const codec =
    (codecId && entries.find((s) => s.id === codecId)?.mimeType?.split('/')[1]) ||
    'unknown';

  const rttMs = candidatePair?.currentRoundTripTime
    ? candidatePair.currentRoundTripTime * 1000
    : 0;
  const jitterMs = (inbound.jitter ?? 0) * 1000;
  const packetsLost = inbound.packetsLost ?? 0;
  const packetsReceived = inbound.packetsReceived ?? 0;
  const denom = packetsLost + packetsReceived;
  const packetLoss = denom > 0 ? packetsLost / denom : 0;
  const bitrateKbps = inbound.bytesReceived
    ? Math.round((inbound.bytesReceived * 8) / 1000 / Math.max(1, inbound._intervalSec ?? 1))
    : 0;

  return {
```

```
      rttMs: Math.round(rttMs),
      jitterMs: Math.round(jitterMs),
      packetLoss: Number(packetLoss.toFixed(4)),
      bitrateKbps,
      codec,
      fps: inbound.framesPerSecond ?? undefined,
    };
  }
```

`apps/desktop/src/renderer/src/services/diagnosticsReporter.ts` — SAFE_DIRECT_COPY

```
import {
  computeQualityScore,
  type DiagnosticSampleInput,
  type QualityMetrics,
} from '@remotedesk/shared';

/** Injected transport so tests don't hit the network. */
export type SampleSink = (sample: DiagnosticSampleInput) => Promise<void>;
export type MetricsSource = () => Promise<QualityMetrics | null>;

export interface ReporterOptions {
  intervalMs?: number;
  deviceId?: string | null;
  /** Optional preview callback for a live HUD (uses the shared score). */
  onSample?: (sample: DiagnosticSampleInput & { qualityScore: number }) => void;
}

/**
 * Periodically samples WebRTC metrics and POSTs them to the API. FAILS SILENTLY:
 * a sink/network error is swallowed (logged at debug) so diagnostics never break
 * a live session.
 */
export class DiagnosticsReporter {
  private timer: ReturnType<typeof setInterval> | null = null;
  private readonly intervalMs: number;

  constructor(
    private readonly sessionId: string,
    private readonly source: MetricsSource,
    private readonly sink: SampleSink,
    private readonly opts: ReporterOptions = {},
  ) {
    this.intervalMs = opts.intervalMs ?? 10_000;
  }

  start(): void {
    if (this.timer) return;
    this.timer = setInterval(() => void this.tick(), this.intervalMs);
  }

  stop(): void {
    if (this.timer) clearInterval(this.timer);
    this.timer = null;
  }

  /** One sample cycle. Exposed for tests + manual flush. */
```

```
    async tick(): Promise<void> {
      let metrics: QualityMetrics | null = null;
      try {
        metrics = await this.source();
      } catch {
        return; // no metrics available this cycle; fail silently
      }
      if (!metrics) return;

      const sample: DiagnosticSampleInput = {
        ...metrics,
        sessionId: this.sessionId,
        deviceId: this.opts.deviceId ?? null,
        capturedAt: new Date().toISOString(),
      };
      this.opts.onSample?.({ ...sample, qualityScore: computeQualityScore(metrics) });

      try {
        await this.sink(sample);
      } catch {
        // Swallow: diagnostics must never interrupt the session.
      }
    }
  }

  /** Default HTTP sink. Throws on non-2xx so the reporter swallows it upstream. */
  export function httpSampleSink(baseUrl: string, authToken: string): SampleSink {
    return async (sample) => {
      const res = await fetch(`${baseUrl}/api/diagnostics/samples`, {
        method: 'POST',
        headers: {
          'content-type': 'application/json',
          authorization: `Bearer ${authToken}`,
        },
        body: JSON.stringify(sample),
      });
      if (!res.ok) throw new Error(`sample post failed: ${res.status}`);
    };
  }
```

`apps/desktop/src/renderer/src/services/supportBundleClient.ts` — SAFE_DIRECT_COPY

```
  import {
    buildSupportBundleMetadata,
    type SupportBundleDto,
  } from '@remotedesk/shared';

  /**
   * Builds a redacted support bundle client-side (defense in depth — the server
   * redacts again authoritatively) and posts it. NO SECRETS are gathered: callers
   * pass a context map, and secret-bearing keys are scrubbed before send.
   */
  export interface SupportBundleRequest {
    appVersion: string;
    osVersion: string;
    sessionId?: string | null;
    deviceId?: string | null;
```

```
  context?: Record<string, string>;
  redactEmails?: boolean;
}

export class SupportBundleClient {
  constructor(
    private readonly baseUrl: string,
    private readonly authToken: string,
  ) {}

  async create(req: SupportBundleRequest): Promise<SupportBundleDto> {
    // Local redaction first so nothing sensitive even leaves the device.
    const { metadata } = buildSupportBundleMetadata({
      appVersion: req.appVersion,
      osVersion: req.osVersion,
      sessionId: req.sessionId ?? null,
      deviceId: req.deviceId ?? null,
      context: req.context,
      redactEmails: req.redactEmails,
    });
    const res = await fetch(`${this.baseUrl}/api/diagnostics/support-bundles`, {
      method: 'POST',
      headers: {
        'content-type': 'application/json',
        authorization: `Bearer ${this.authToken}`,
      },
      body: JSON.stringify({
        appVersion: metadata.appVersion,
        osVersion: metadata.osVersion,
        sessionId: metadata.sessionId,
        deviceId: metadata.deviceId,
        context: metadata.context,
        redactEmails: metadata.redactEmails,
      }),
    });
    if (!res.ok) throw new Error(`support bundle create failed: ${res.status}`);
    return res.json();
  }
}
```

`apps/desktop/src/renderer/src/session/sessionSetup.ts` — REVIEW_REQUIRED (patch)

> Wire the reporter into your existing session/WebRTC flow. `peerConnection` is the live `RTCPeerConnection` from the session engine in earlier batches.
> **Native input execution stays disabled — this patch only reads stats.**

```
// ...existing imports...
import { DiagnosticsReporter, httpSampleSink } from '../services/diagnosticsReporter';
import { collectRtcMetrics } from '../services/rtcStatsCollector';

// --- after the peer connection is established and sessionId is known ---
const diagnosticsReporter = new DiagnosticsReporter(
  sessionId,
  () => collectRtcMetrics(peerConnection), // read-only getStats()
  httpSampleSink(apiBase, authToken),
  {
    intervalMs: 10_000,
```

```
    deviceId,
    onSample: (s) => updateQualityHud(s.qualityScore), // your existing HUD, optional
  },
);
diagnosticsReporter.start();

// --- on session teardown ---
session.onClose(() => diagnosticsReporter.stop());
```

Part E — Web: diagnostics dashboard + per-session view

`apps/web/app/dashboard/diagnostics/diagnosticsApi.ts` — SAFE_DIRECT_COPY

```
import type {
  DiagnosticSampleDto,
  SessionQualitySummaryDto,
  SupportBundleDto,
} from '@remotedesk/shared';
import { api } from '../../../src/api/apiClient';

export const diagnosticsApi = {
  session: (sessionId: string) =>
    api.get<{ summary: SessionQualitySummaryDto; samples: DiagnosticSampleDto[] }>(
      `/diagnostics/sessions/${sessionId}`,
    ),
  createBundle: (input: {
    appVersion: string;
    osVersion: string;
    sessionId?: string | null;
    context?: Record<string, string>;
    redactEmails?: boolean;
  }) => api.post<SupportBundleDto>('/diagnostics/support-bundles', input),
  getBundle: (bundleId: string) =>
    api.get<SupportBundleDto>(`/diagnostics/support-bundles/${bundleId}`),
};
```

`apps/web/app/dashboard/diagnostics/QualityBadge.tsx` — SAFE_DIRECT_COPY

```
'use client';

import React from 'react';
import { qualityBand } from '@remotedesk/shared';
import styles from './diagnostics.module.css';

export function QualityBadge({ score }: { score: number }) {
  const band = qualityBand(score);
  return (
    <span className={styles.badge} data-band={band} title={`Quality ${score}/100`}>
      {score} · {band}
    </span>
  );
}
```

`apps/web/app/dashboard/diagnostics/page.tsx` — SAFE_DIRECT_COPY

```
'use client';

import React, { useState } from 'react';
import type { SupportBundleDto } from '@remotedesk/shared';
import { diagnosticsApi } from './diagnosticsApi';
import styles from './diagnostics.module.css';

export default function DiagnosticsPage() {
  const [sessionId, setSessionId] = useState('');
  const [redactEmails, setRedactEmails] = useState(true);
  const [bundle, setBundle] = useState<SupportBundleDto | null>(null);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const createBundle = async () => {
    setBusy(true);
    setError(null);
    try {
      const created = await diagnosticsApi.createBundle({
        appVersion: process.env.NEXT_PUBLIC_APP_VERSION ?? 'web',
        osVersion: typeof navigator !== 'undefined' ? navigator.userAgent : 'unknown',
        sessionId: sessionId || null,
        context: {}, // dashboard collects no device context; desktop bundles do
        redactEmails,
      });
      setBundle(created);
    } catch {
      setError('Could not create support bundle.');
    } finally {
      setBusy(false);
    }
  };

  const downloadMetadata = () => {
    if (!bundle) return;
    const blob = new Blob([JSON.stringify(bundle, null, 2)], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `support-bundle-${bundle.id}.json`;
    a.click();
    URL.revokeObjectURL(url);
  };

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Diagnostics & Support Bundles</h1>
        <p className={styles.sub}>
          Create a redacted support bundle. Tokens, passwords, cookies, auth
          headers and API keys are always scrubbed; emails optionally.
        </p>
      </header>

      {error && <div className={styles.error}>{error}</div>}
```

```
      <section className={styles.panel}>
        <h2>New support bundle</h2>
        <label className={styles.field}>
          <span>Session ID (optional)</span>
          <input value={sessionId} onChange={(e) => setSessionId(e.target.value)} placeholder="sess_…" />
        </label>
        <label className={styles.checkbox}>
          <input type="checkbox" checked={redactEmails} onChange={(e) => setRedactEmails(e.target.checked)} />
          <span>Redact email addresses</span>
        </label>
        <button type="button" disabled={busy} className={styles.btnPrimary} onClick={createBundle}>
          {busy ? 'Creating…' : 'Create bundle'}
        </button>
      </section>

      {bundle && (
        <section className={styles.panel}>
          <h2>Bundle {bundle.id}</h2>
          <p>Created {new Date(bundle.createdAt).toLocaleString()}</p>
          <p>
            Redactions: {bundle.redactionReport.totalRedactions}
            {bundle.redactionReport.emailsRedacted ? ' (emails scrubbed)' : ''}
          </p>
          {bundle.redactionReport.redactedKeys.length > 0 && (
            <p className={styles.muted}>
              Scrubbed keys: {bundle.redactionReport.redactedKeys.join(', ')}
            </p>
          )}
          <button type="button" className={styles.btnSecondary} onClick={downloadMetadata}>
            Download metadata (JSON)
          </button>
        </section>
      )}
    </main>
  );
}
```

`apps/web/app/dashboard/diagnostics/diagnostics.module.css` — SAFE_DIRECT_COPY

```
.page { padding: 24px; max-width: 1080px; margin: 0 auto; }
.header h1 { margin: 0 0 4px; font-size: 22px; }
.sub { color: var(--muted, #6b7280); margin: 0 0 20px; }
.error { background: #fef2f2; color: #991b1b; padding: 10px 14px; border-radius: 8px; margin-bottom: 16px; }
.panel { border: 1px solid var(--border, #e5e7eb); border-radius: 12px; padding: 16px 18px; margin-bottom: 16px;
background: var(--surface, #fff); }
.panel h2 { font-size: 15px; margin: 0 0 12px; }
.field { display: flex; flex-direction: column; gap: 4px; margin-bottom: 12px; font-size: 13px; }
.field input { padding: 8px 10px; border: 1px solid var(--border, #e5e7eb); border-radius: 8px; }
.checkbox { display: flex; align-items: center; gap: 8px; margin-bottom: 12px; font-size: 13px; }
.muted { color: var(--muted, #6b7280); font-size: 12px; }
.btnPrimary { background: #4f46e5; color: #fff; border: none; padding: 8px 14px; border-radius: 8px; cursor: pointer; }
.btnSecondary { background: #fff; color: #4f46e5; border: 1px solid #c7d2fe; padding: 8px 14px; border-radius: 8px;
cursor: pointer; }
.btnPrimary:disabled { opacity: 0.6; cursor: default; }
.table { width: 100%; border-collapse: collapse; font-size: 13px; }
.table th, .table td { text-align: left; padding: 8px 10px; border-bottom: 1px solid var(--border, #e5e7eb); }
.table th { color: var(--muted, #6b7280); font-weight: 600; }
```

```
.badge { font-size: 12px; padding: 2px 10px; border-radius: 999px; background: #f3f4f6; color: #374151; }
.badge[data-band='excellent'] { background: #ecfdf5; color: #065f46; }
.badge[data-band='good'] { background: #eff6ff; color: #1d4ed8; }
.badge[data-band='fair'] { background: #fffbeb; color: #92400e; }
.badge[data-band='poor'] { background: #fef2f2; color: #991b1b; }
.summaryGrid { display: grid; grid-template-columns: repeat(auto-fit, minmax(120px, 1fr)); gap: 12px; }
.metric { border: 1px solid var(--border, #e5e7eb); border-radius: 10px; padding: 10px 12px; }
.metric .label { font-size: 11px; color: var(--muted, #6b7280); }
.metric .value { font-size: 18px; font-weight: 600; }
```

`apps/web/app/dashboard/sessions/[sessionId]/diagnostics/SampleTable.tsx` — SAFE_DIRECT_COPY

```
'use client';

import React from 'react';
import type { DiagnosticSampleDto } from '@remotedesk/shared';
import { QualityBadge } from '../../../diagnostics/QualityBadge';
import styles from '../../../diagnostics/diagnostics.module.css';

export function SampleTable({ samples }: { samples: DiagnosticSampleDto[] }) {
  if (samples.length === 0) {
    return <p className={styles.muted}>No samples recorded for this session yet.</p>;
  }
  return (
    <table className={styles.table}>
      <thead>
        <tr>
          <th>Time</th>
          <th>Quality</th>
          <th>RTT (ms)</th>
          <th>Jitter (ms)</th>
          <th>Loss</th>
          <th>Bitrate (kbps)</th>
          <th>Codec</th>
        </tr>
      </thead>
      <tbody>
        {samples.map((s) => (
          <tr key={s.id}>
            <td>{new Date(s.capturedAt).toLocaleTimeString()}</td>
            <td><QualityBadge score={s.qualityScore} /></td>
            <td>{s.rttMs}</td>
            <td>{s.jitterMs}</td>
            <td>{(s.packetLoss * 100).toFixed(2)}%</td>
            <td>{s.bitrateKbps}</td>
            <td>{s.codec}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

`apps/web/app/dashboard/sessions/[sessionId]/diagnostics/page.tsx` — SAFE_DIRECT_COPY

```
'use client';
```

```
import React, { useCallback, useEffect, useState } from 'react';
import { useParams } from 'next/navigation';
import type { DiagnosticSampleDto, SessionQualitySummaryDto } from '@remotedesk/shared';
import { diagnosticsApi } from '../../../diagnostics/diagnosticsApi';
import { QualityBadge } from '../../../diagnostics/QualityBadge';
import { SampleTable } from './SampleTable';
import styles from '../../../diagnostics/diagnostics.module.css';

export default function SessionDiagnosticsPage() {
  const params = useParams<{ sessionId: string }>();
  const sessionId = params.sessionId;
  const [summary, setSummary] = useState<SessionQualitySummaryDto | null>(null);
  const [samples, setSamples] = useState<DiagnosticSampleDto[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const load = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const data = await diagnosticsApi.session(sessionId);
      setSummary(data.summary);
      setSamples(data.samples);
    } catch {
      setError('Could not load session diagnostics.');
    } finally {
      setLoading(false);
    }
  }, [sessionId]);

  useEffect(() => {
    void load();
  }, [load]);

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Session diagnostics</h1>
        <p className={styles.sub}>{sessionId}</p>
      </header>

      {error && <div className={styles.error}>{error}</div>}
      {loading ? (
        <p className={styles.muted}>Loading…</p>
      ) : (
        <>
          {summary && (
            <section className={styles.panel}>
              <h2>Summary ({summary.sampleCount} samples)</h2>
              <div className={styles.summaryGrid}>
                <div className={styles.metric}>
                  <div className="label">Avg quality</div>
                  <div className="value"><QualityBadge score={Math.round(summary.avgQualityScore)} /></div>
                </div>
                <div className={styles.metric}>
                  <div className="label">Worst</div>
                  <div className="value">{summary.worstQualityScore}</div>
```

```
                </div>
                <div className={styles.metric}>
                  <div className="label">Avg RTT</div>
                  <div className="value">{summary.avgRttMs} ms</div>
                </div>
                <div className={styles.metric}>
                  <div className="label">Avg jitter</div>
                  <div className="value">{summary.avgJitterMs} ms</div>
                </div>
                <div className={styles.metric}>
                  <div className="label">Avg loss</div>
                  <div className="value">{(summary.avgPacketLoss * 100).toFixed(2)}%</div>
                </div>
                <div className={styles.metric}>
                  <div className="label">Avg bitrate</div>
                  <div className="value">{summary.avgBitrateKbps} kbps</div>
                </div>
              </div>
            </section>
          )}
          <section className={styles.panel}>
            <h2>Samples</h2>
            <SampleTable samples={samples} />
          </section>
        </>
      )}
    </main>
  );
}
```

`apps/web/app/dashboard/page.tsx` — REVIEW_REQUIRED (patch)

> Add a link into your existing dashboard grid.

```
// ...existing imports...
import Link from 'next/link';

// ...inside the dashboard nav/grid, alongside the other section links:
<Link href="/dashboard/diagnostics" className="dashboard-card">
  <h3>Diagnostics</h3>
  <p>Session quality telemetry and redacted support bundles.</p>
</Link>
```

Part F — Tests

> All tests import only real shared/copied files. No REVIEW_REQUIRED-only imports.

`tests/diagnostics/qualityScore.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { computeQualityScore, qualityBand, type QualityMetrics } from '@remotedesk/shared';

const perfect: QualityMetrics = { rttMs: 0, jitterMs: 0, packetLoss: 0, bitrateKbps: 3000, codec: 'VP8' };
const awful: QualityMetrics = { rttMs: 400, jitterMs: 150, packetLoss: 0.2, bitrateKbps: 0, codec: 'VP8' };

describe('computeQualityScore', () => {
```

```
  it('scores a perfect connection at or near 100', () => {
    expect(computeQualityScore(perfect)).toBe(100);
  });

  it('scores an awful connection at or near 0', () => {
    expect(computeQualityScore(awful)).toBe(0);
  });

  it('is monotonic: more packet loss never raises the score', () => {
    const a = computeQualityScore({ ...perfect, packetLoss: 0.02 });
    const b = computeQualityScore({ ...perfect, packetLoss: 0.08 });
    expect(a).toBeGreaterThanOrEqual(b);
  });

  it('treats NaN metrics as worst-case (fails toward caution)', () => {
    const score = computeQualityScore({ ...perfect, rttMs: NaN });
    expect(score).toBeLessThan(100);
  });

  it('maps scores to bands', () => {
    expect(qualityBand(95)).toBe('excellent');
    expect(qualityBand(70)).toBe('good');
    expect(qualityBand(50)).toBe('fair');
    expect(qualityBand(10)).toBe('poor');
  });
});
```

`tests/diagnostics/redaction.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { redactContext, assertNoSensitiveKeys, REDACTION_MASK } from '@remotedesk/shared';

describe('redactContext', () => {
  it('scrubs tokens, passwords, cookies, auth headers and api keys', () => {
    const { redacted, report } = redactContext(
      {
        Authorization: 'Bearer abc.def',
        Cookie: 'sid=123',
        access_token: 'xyz',
        password: 'hunter2',
        api_key: 'k-1',
        note: 'safe value',
      },
      false,
    );
    expect(redacted.Authorization).toBe(REDACTION_MASK);
    expect(redacted.Cookie).toBe(REDACTION_MASK);
    expect(redacted.access_token).toBe(REDACTION_MASK);
    expect(redacted.password).toBe(REDACTION_MASK);
    expect(redacted.api_key).toBe(REDACTION_MASK);
    expect(redacted.note).toBe('safe value');
    expect(report.totalRedactions).toBe(5);
  });

  it('redacts emails only when requested', () => {
    const off = redactContext({ msg: 'reach me at a@b.com' }, false);
    expect(off.redacted.msg).toContain('a@b.com');
```

```
    expect(off.report.emailsRedacted).toBe(false);

    const on = redactContext({ msg: 'reach me at a@b.com' }, true);
    expect(on.redacted.msg).not.toContain('a@b.com');
    expect(on.report.emailsRedacted).toBe(true);
  });

  it('assertNoSensitiveKeys throws on an unredacted secret', () => {
    expect(() => assertNoSensitiveKeys({ token: 'raw' })).toThrow();
    expect(() => assertNoSensitiveKeys({ token: REDACTION_MASK })).not.toThrow();
  });
});
```

`tests/diagnostics/supportBundle.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import { buildSupportBundleMetadata, REDACTION_MASK } from '@remotedesk/shared';

describe('buildSupportBundleMetadata', () => {
  it('redacts secret context before producing metadata', () => {
    const { metadata, report } = buildSupportBundleMetadata({
      appVersion: '1.2.3',
      osVersion: 'macOS 14',
      context: { Authorization: 'Bearer t', region: 'eu' },
    });
    expect(metadata.context.Authorization).toBe(REDACTION_MASK);
    expect(metadata.context.region).toBe('eu');
    expect(report.totalRedactions).toBe(1);
  });

  it('never retains secrets and never throws on clean input', () => {
    const { metadata } = buildSupportBundleMetadata({
      appVersion: '1.0.0',
      osVersion: 'Win 11',
      context: { build: 'abc' },
      redactEmails: true,
    });
    expect(metadata.context.build).toBe('abc');
    expect(metadata.redactEmails).toBe(true);
  });

  it('defaults sessionId/deviceId to null', () => {
    const { metadata } = buildSupportBundleMetadata({ appVersion: 'v', osVersion: 'o' });
    expect(metadata.sessionId).toBeNull();
    expect(metadata.deviceId).toBeNull();
  });
});
```

`tests/diagnostics/api-contract.test.ts` — SAFE_DIRECT_COPY

```
import { describe, it, expect } from 'vitest';
import {
  diagnosticSampleInputSchema,
  supportBundleInputSchema,
} from '@remotedesk/shared';
import { summarize, scoreSample } from '../../apps/api/src/lib/diagnostics';
import type { DiagnosticSampleDto } from '@remotedesk/shared';
```

```
describe('diagnostics API contract', () => {
  it('accepts a valid sample payload', () => {
    const parsed = diagnosticSampleInputSchema.parse({
      sessionId: 'sess_1',
      rttMs: 40,
      jitterMs: 5,
      packetLoss: 0.01,
      bitrateKbps: 2200,
      codec: 'VP8',
      capturedAt: new Date().toISOString(),
    });
    expect(parsed.sessionId).toBe('sess_1');
  });

  it('rejects packet loss outside 0..1', () => {
    expect(() =>
      diagnosticSampleInputSchema.parse({
        sessionId: 'sess_1',
        rttMs: 40,
        jitterMs: 5,
        packetLoss: 2,
        bitrateKbps: 2200,
        codec: 'VP8',
        capturedAt: new Date().toISOString(),
      }),
    ).toThrow();
  });

  it('accepts a valid support bundle payload', () => {
    const parsed = supportBundleInputSchema.parse({
      appVersion: '1.0.0',
      osVersion: 'Linux',
      context: { k: 'v' },
      redactEmails: true,
    });
    expect(parsed.appVersion).toBe('1.0.0');
  });

  it('scoreSample attaches a derived score', () => {
    const scored = scoreSample({ rttMs: 0, jitterMs: 0, packetLoss: 0, bitrateKbps: 3000, codec: 'VP8' });
    expect(scored.qualityScore).toBe(100);
  });

  it('summarize folds samples into averages + worst', () => {
    const base: Omit<DiagnosticSampleDto, 'qualityScore' | 'rttMs'> = {
      id: 'x', sessionId: 's', deviceId: null, jitterMs: 5, packetLoss: 0.01,
      bitrateKbps: 2000, codec: 'VP8', capturedAt: new Date().toISOString(),
    };
    const summary = summarize('s', [
      { ...base, id: 'a', rttMs: 20, qualityScore: 90 },
      { ...base, id: 'b', rttMs: 60, qualityScore: 70 },
    ]);
    expect(summary.sampleCount).toBe(2);
    expect(summary.avgQualityScore).toBe(80);
    expect(summary.worstQualityScore).toBe(70);
```

```
  });
});
```

Part G — Docs

`docs/diagnostics-and-support-bundles.md` — SAFE_DIRECT_COPY

```
# Diagnostics, Session Quality Telemetry & Support Bundles

Real WebRTC session-quality telemetry plus a redacted support-bundle foundation.
**No native input execution is enabled by this pack. No live third-party
integrations. This is diagnostics scaffolding, not production-ready support tooling.**

## Quality score
`computeQualityScore(metrics)` in `@remotedesk/shared` returns a 0..100 score from
a weighted penalty model: packet loss (0.35), RTT (0.30), jitter (0.20),
bitrate (0.15). Unknown/NaN metrics score toward the worst case. The SAME
function runs on desktop (HUD preview), API (persisted score), and web (display),
so the number never differs across surfaces. `qualityBand(score)` buckets it into
excellent / good / fair / poor for coloring.

## Data model
- **DiagnosticSample** — one telemetry reading, linked to `Session` (cascade) and
  `Device` (set-null). Stores RTT, jitter, loss, bitrate, codec, fps, derived score, capturedAt.
- **SessionQualitySummary** — rolled-up averages + worst score per session, refreshed on each insert.
- **SupportBundle** — redacted, content-light metadata + a redaction report. Never stores secrets.

## Endpoints
- `POST /api/diagnostics/samples` — record a sample (server derives the score, refreshes the summary).
- `GET  /api/diagnostics/sessions/:sessionId` — `{ summary, samples }` for a session.
- `POST /api/diagnostics/support-bundles` — create a bundle; **redaction runs before persistence**.
- `GET  /api/diagnostics/support-bundles/:bundleId` — fetch redacted bundle metadata (org-scoped).

## Desktop reporting
`DiagnosticsReporter` samples `RTCPeerConnection.getStats()` every 10s (read-only;
no input execution) and POSTs the result. It **fails silently** — any source or
network error is swallowed so diagnostics never interrupt a live session.

## Redaction (mandatory)
`redactContext` scrubs keys matching: authorization, cookie/set-cookie, token,
password, secret, api key, access key, refresh token, bearer, private key. Emails
in values are scrubbed only when `redactEmails` is true. `buildSupportBundleMetadata`
redacts, then `assertNoSensitiveKeys` is a defense-in-depth backstop. The desktop
client also redacts locally before sending, so secrets never leave the device.
```

Part H — Handoff meta (doNotMerge)

`generated-remotedesk-pack12-merge-summary.md` — doNotMerge

```
# RemoteDesk Pack 12 — Merge Summary

**Pack:** Diagnostics, Session Quality Telemetry & Support Bundles
```

```
**Files:** 37 (28 SAFE_DIRECT_COPY, 6 REVIEW_REQUIRED, 3 doNotMerge meta)

## Build order
1. `packages/shared` — diagnostics/* then patch shared `index.ts` + add the `SUPPORT_BUNDLE_NOT_FOUND` error code; `pnpm --filter @remotedesk/shared build`.
2. `apps/api` — patch `schema.prisma` (3 models + 2 back-relations), run the `0012_diagnostics` migration + `prisma generate`, drop in lib/repo/services/routes, patch `server.ts`.
3. `apps/web` — drop in `dashboard/diagnostics/*` and `sessions/[sessionId]/diagnostics/*`, patch the dashboard link.
4. `apps/desktop` — drop in the three services, patch `session/sessionSetup.ts` to start the reporter (read-only getStats).
5. Tests — `pnpm -r test`.

## REVIEW_REQUIRED (hand-merge)
- `packages/shared/src/index.ts` — add diagnostics barrel.
- `packages/shared/src/errors/errorCodes.ts` — add SUPPORT_BUNDLE_NOT_FOUND.
- `apps/api/prisma/schema.prisma` — add 3 models + back-relations on Session/Device.
- `apps/api/src/server.ts` — mount /api/diagnostics.
- `apps/web/app/dashboard/page.tsx` — add diagnostics link.
- `apps/desktop/src/renderer/src/session/sessionSetup.ts` — start/stop the reporter.

## Known limitations
- Support bundles store **metadata only** — no file/log attachment upload yet (foundation only).
- `rtcStatsCollector` bitrate uses a simplified interval; wire a delta-based sampler for precise kbps.
- Per-session diagnostics route is auth-only, not org-scoped; add org/session ownership checks before prod.
- Web dashboard bundle UI collects no device context (browser can't); rich context comes from the desktop client.
- No live third-party integrations; nothing here is production-ready support tooling.

## Safety
- Native input execution remains DISABLED — the desktop patch only reads getStats().
- Redaction is mandatory and runs before persistence (server) and before send (desktop).

## Verify
- `pnpm -r typecheck && pnpm -r test`
- Manual: run a session, confirm DiagnosticSample rows + a SessionQualitySummary appear.
- Manual: create a bundle with an `Authorization` context key, confirm it is stored as [REDACTED].
```

`generated-remotedesk-pack12-code-review.md` — doNotMerge

```
# RemoteDesk Pack 12 — Code Review Notes

## Strengths
- One scoring function shared by desktop, API, and web — no drift.
- Redaction is pure, table-driven, and double-applied (client + server) with an assert backstop.
- Reporter fails silently and never blocks the session; transport is injected for testability.
- Summary math is pure (`summarize`) and unit-tested without a DB.
- Tests import only real generated/copied files — no REVIEW_REQUIRED-only imports.

## Reviewer checklist
- [ ] `Session` / `Device` model names + id fields match the migration FKs.
- [ ] `apiClient` exposes `get`/`post`. Confirm base path strips/keeps `/api` consistently.
- [ ] Add org/session ownership check to `GET /diagnostics/sessions/:sessionId` before prod.
- [ ] Reporter interval (10s) acceptable vs sample volume / retention.
- [ ] `rtcStatsCollector.parseStats` matches your browser/Electron getStats shape; adjust bitrate interval.
- [ ] Confirm `prisma.$queryRaw` not needed; Json columns store redacted metadata only.

## Security
- No secrets persisted: redaction precedes every write; `assertNoSensitiveKeys` throws if one survives.
```

```
- Email redaction is opt-in per request and reported in the redaction report.
- Desktop never gathers secrets; it redacts locally before sending.
- Native input execution untouched and still disabled.
```

`generated-remotedesk-pack12-manifest.json` — doNotMerge


```
{
  "pack": "pack12",
  "title": "Diagnostics, Session Quality Telemetry & Support Bundles",
  "actualFileCount": 37,
  "safeDirectCopy": [
    "packages/shared/src/diagnostics/types.ts",
    "packages/shared/src/diagnostics/qualityScore.ts",
    "packages/shared/src/diagnostics/supportBundle.ts",
    "packages/shared/src/diagnostics/redaction.ts",
    "packages/shared/src/diagnostics/schema.ts",
    "packages/shared/src/diagnostics/index.ts",
    "apps/api/prisma/migrations/0012_diagnostics/migration.sql",
    "apps/api/src/lib/diagnostics.ts",
    "apps/api/src/repositories/diagnosticsRepository.ts",
    "apps/api/src/services/diagnostics/diagnostics.service.ts",
    "apps/api/src/services/diagnostics/supportBundle.service.ts",
    "apps/api/src/routes/diagnostics.routes.ts",
    "apps/web/app/dashboard/diagnostics/page.tsx",
    "apps/web/app/dashboard/diagnostics/diagnosticsApi.ts",
    "apps/web/app/dashboard/diagnostics/QualityBadge.tsx",
    "apps/web/app/dashboard/diagnostics/diagnostics.module.css",
    "apps/web/app/dashboard/sessions/[sessionId]/diagnostics/page.tsx",
    "apps/web/app/dashboard/sessions/[sessionId]/diagnostics/SampleTable.tsx",
    "apps/desktop/src/renderer/src/services/diagnosticsReporter.ts",
    "apps/desktop/src/renderer/src/services/supportBundleClient.ts",
    "apps/desktop/src/renderer/src/services/rtcStatsCollector.ts",
    "tests/diagnostics/qualityScore.test.ts",
    "tests/diagnostics/redaction.test.ts",
    "tests/diagnostics/supportBundle.test.ts",
    "tests/diagnostics/api-contract.test.ts",
    "docs/diagnostics-and-support-bundles.md"
  ],
  "reviewRequired": [
    "packages/shared/src/index.ts",
    "packages/shared/src/errors/errorCodes.ts",
    "apps/api/prisma/schema.prisma",
    "apps/api/src/server.ts",
    "apps/web/app/dashboard/page.tsx",
    "apps/desktop/src/renderer/src/session/sessionSetup.ts"
  ],
  "doNotMerge": [
    "generated-remotedesk-pack12-merge-summary.md",
    "generated-remotedesk-pack12-code-review.md",
    "generated-remotedesk-pack12-manifest.json"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["zod", "@prisma/client", "express", "next", "react", "vitest"],
    "assumedFromPriorBatches": [
      "@remotedesk/shared AppError + ErrorCodes",
      "apps/api db/prisma client",
```

```
        "apps/api middleware: requireAuth, orgContext",
        "apps/web src/api/apiClient (api.get/post)",
        "apps/desktop session engine exposing a live RTCPeerConnection + sessionId",
        "existing Prisma Session and Device models"
      ]
    },
    "knownLimitations": [
      "Support bundles store redacted metadata only; no log/file attachment upload yet.",
      "rtcStatsCollector bitrate uses a simplified interval; add a delta sampler for precise kbps.",
      "GET /diagnostics/sessions/:sessionId is auth-only; add org/session ownership checks before prod.",
      "Web bundle UI gathers no device context; rich context comes from the desktop client.",
      "No live third-party integrations; this is diagnostics scaffolding, not production support tooling.",
      "Native input execution remains disabled by design."
    ],
    "estimatedCompletionAfterCarefulMerge": "60-66%"
  }
```

Pack 12 done: 37 files. Real WebRTC quality telemetry (RTT, jitter, packet loss, bitrate, codec) sampled read-only every 10s, a single shared 0..100 quality score used identically on desktop/API/web, persisted samples linked to Session + Device, a rolled-up per-session summary, and a redacted support-bundle foundation that scrubs tokens / passwords / cookies / auth headers / API keys (emails opt-in) **before** anything hits the DB.

The clean bit: redaction is pure, table-driven, and applied twice (desktop before send, server before persist) with an `assertNoSensitiveKeys` backstop, so secrets can't reach storage. The reporter fails silently and never blocks a live session, and native input execution stays fully disabled.

Next number = Pack 13 (support-bundle log/file attachment upload + retention), or org-scoped diagnostics access control. Bolo.